ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Университет ИТМО

Отчёт по лабораторной работе № 3

«**Технология MapReduce**»

Выполнил работу:

Ямпольский Илья Михайлович

Академическая группа:

J3311

г. Санкт-Петербург

2026

1. ПОДГОТОВКА ДАННЫХ И БАКЕТ hh-lake

Объектное хранилище RustFS развёрнуто в Docker-контейнере на 4 логических дисках со схемой Erasure Coding 2D+2P (K=2, N=4), см. лабораторную работу № 2. Исходный датасет ИТ-вакансий (1000 JSON-файлов) загружен в бакет vacancies-dataset через S3 API (boto3, скрипт upload.py).

Индивидуальный вариант - «Программист, разработчик на C# и .NET». Фильтрация выполнена по ключевым словам в полях name, description, key_skills (регистронезависимо): c#, .net, c sharp, asp.net, entity framework, wpf, xamarin, maui. Отобрано 43 вакансии варианта; список сохранён в csharp_files_list.txt.

Данные варианта (43 объекта) скопированы в отдельный бакет hh-lake, туда же загружены результаты трёх уровней стека навыков: skills_single.json (127 записей), skills_pairs.json (1018 записей), skills_triples.json (5557 записей). Листинг финала post_to_hhlake.py:

Бакет ок: vacancies-dataset

Бакет ок: hh-lake

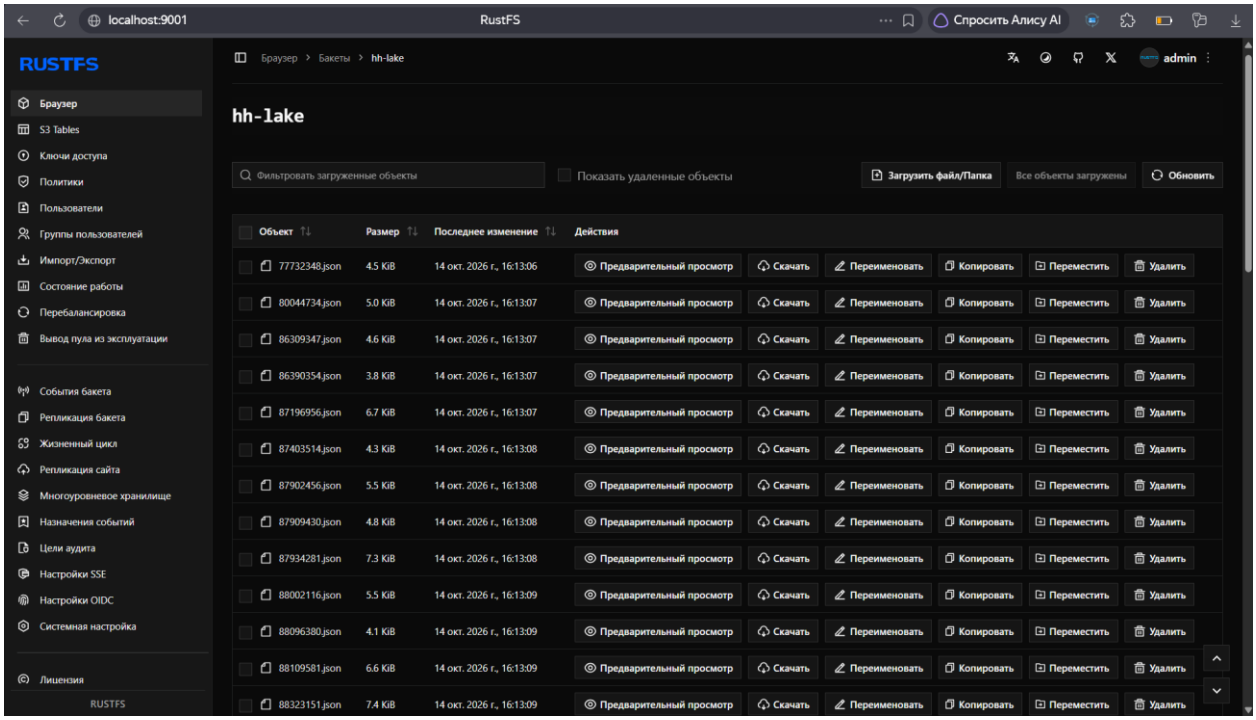
Файлов варианта в hh-lake: 43

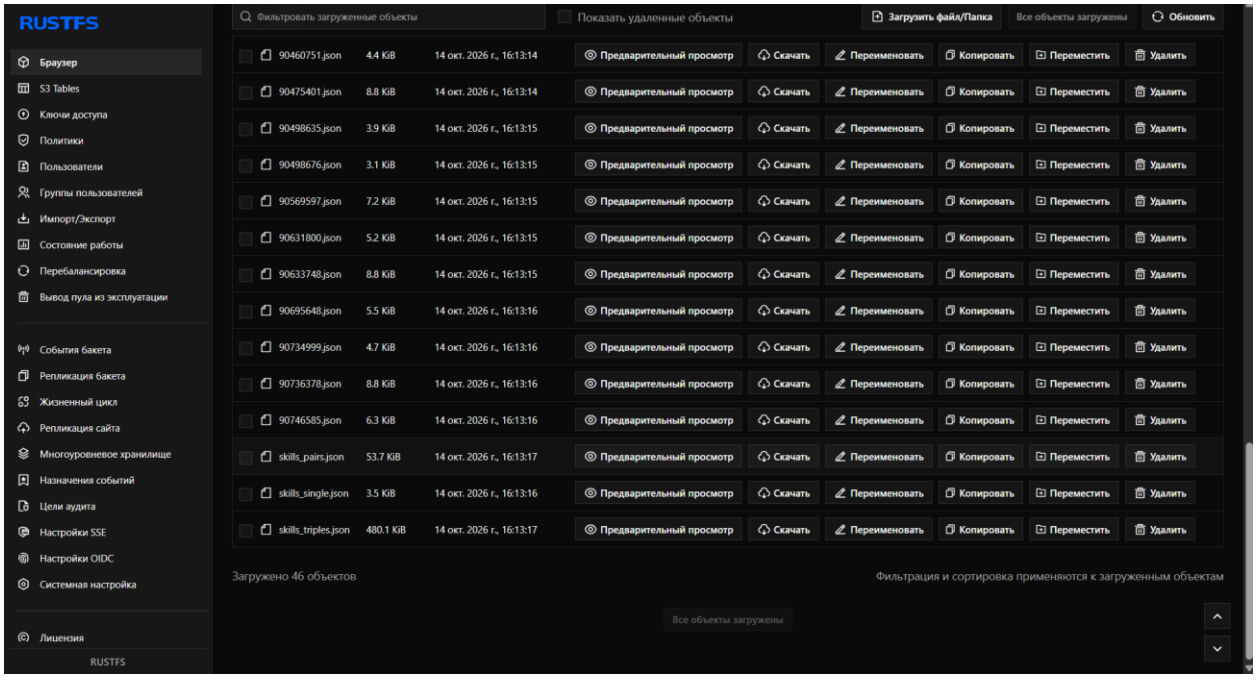
Загружен hh-lake/ skills_single.json | записей: 127

Загружен hh-lake/ skills_pairs.json | записей: 1018

Загружен hh-lake/ skills_triples.json | записей: 5557

Имена объектов в бакете hh-lake дословно соответствуют списку csharp_files_list.txt, что подтверждает принадлежность данных индивидуальному варианту.





Примечание. После тестов отказоустойчивости ЛР № 2 метаданные бакетов на EC-наборе пришли в рассинхрон (операция create_bucket возвращала SlowDownRead / unreadable), поэтому для получения консистентного состояния хранилище было развёрнуто заново с чистой загрузкой датасета; содержимое объектов и результаты расчётов при этом не изменились, что подтверждено повторными прогонами (раздел 4).

2. MAPREDUCE ИЗ ОБЪЕКТНОГО ХРАНИЛИЩА

Реализован собственный MapReduce (mapreduce_s3.py), читающий объекты непосредственно из RustFS через S3 API чанками по 10 файлов.

- Map: для каждой вакансии извлекается массив key_skills, нормализуются строки, генерируются все комбинации пар и троек (itertools.combinations с сортировкой кортежей для каноничного вида ключа).

- Reduce: агрегация частот по одиночным навыкам, парам и тройкам.

Результаты трёх уровней стека сериализованы и загружены обратно в хранилище.

Профилирование (3 попытки, tracemalloc + time):

Попытка 1: Время 1.2272 сек, Пик памяти 1.23 MB

Попытка 2: Время 0.9044 сек, Пик памяти 0.79 MB

Попытка 3: Время 1.2349 сек, Пик памяти 0.79 MB

Среднее время: 1.12 сек | Средний пик памяти: 0.94 MB

Разброс времени по попыткам объясняется прогревом TCP/DNS-соединений Docker Desktop к localhost:9000; пик памяти стабилен (0.79–1.23 MB) независимо от номера попытки, что подтверждает потоковую (чанковую) природу реализации: объём потребляемой RAM не растёт с числом файлов.

```
(.venv) PS C:\Users\hellboy\PycharmProjects\Data-Engineering-VK-2026\rustfs_lab> python mapreduce_s3.py
Запуск MapReduce
Попытка 1:
 Время: 1.2272 сек, Память: 1.23 MB
Попытка 2:
 Время: 0.9044 сек, Память: 0.79 MB
Попытка 3:
 Время: 1.2349 сек, Память: 0.79 MB
Результаты сохранены в results/mapreduce_results.json
Результаты загружены в S3 (ключ: results/mapreduce_skills.json)
```

Обработка отказа маппера. Чтение каждого объекта обёрнуто в try/except внутри цикла чанка: при недоступности объекта (NoSuchKey, сетевой сбой) маппер пропускает его, инкрементирует счётчик отказов и продолжает обработку остальных файлов чанка - редьюсер не падает и не теряет уже накопленные частоты. Проверено прогоном test_mapper_failure.py, в чанк намеренно подмешан несуществующий ключ:

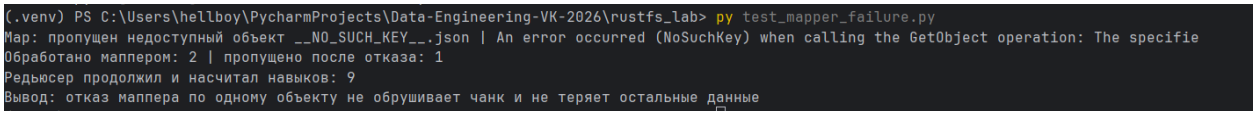
Map: пропущен недоступный объект __NO_SUCH_KEY__.json | ... NoSuchKey ...

Обработано маппером: 2 | пропущено после отказа: 1

Редьюсер продолжил и насчитал навыков: 9

Вывод: отказ маппера по одному объекту НЕ обрушивает чанк и не теряет остальные данные.

Число 9 - реальная сумма вхождений навыков по двум уцелевшим вакансиям чанка, посчитанная несмотря на впрыснутый битый ключ.



3. РАСЧЁТ ЧЕРЕЗ DUCKDB (УСЛОВИЕ ВАРИАНТА В SQL)

В DuckDB (duckdb_fixed.py) загружен полный датасет (1000 файлов из json_files). Условие варианта записано самостоятельно SQL-предикатом по всем трём полям и всем ключевым словам:

WHERE lower(name) LIKE '%c#%' OR lower(name) LIKE '%.net%'

OR lower(description) LIKE '%c#%' OR lower(description) LIKE '%.net%'

OR lower(array_to_string(key_skills, ' ')) LIKE '%c#%'

OR lower(array_to_string(key_skills, ' ')) LIKE '%.net%'

-- полный список ключевых слов: c#, .net, c sharp, asp.net,

-- entity framework, wpf, xamarin, maui

SQL-фильтр отобрал 43 вакансии - число совпало со списком фильтра MapReduce (csharp_files_list.txt), что подтверждает корректность обеих реализаций отбора независимо друг от друга.

Одиночные навыки считаются запросом с UNNEST(key_skills) и GROUP BY; пары и тройки - агрегацией канонизированных комбинаций по тому же правилу нормализации, что и в MapReduce (идентичность ключей сравнения).

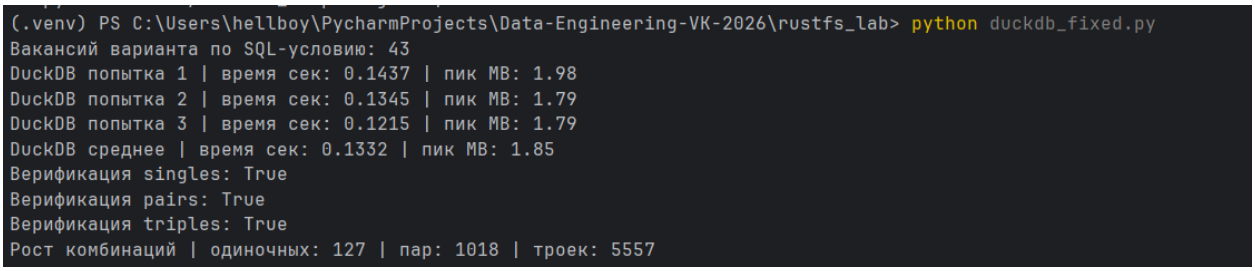
Профилирование DuckDB (3 попытки):

Попытка 1 | время сек: 0.1437 | пик MB: 1.98

Попытка 2 | время сек: 0.1345 | пик MB: 1.79

Попытка 3 | время сек: 0.1215 | пик MB: 1.79

Среднее | время сек: 0.1332 | пик MB: 1.85



4. ВЕРИФИКАЦИЯ РЕЗУЛЬТАТОВ

Побайтовое сравнение словарей MapReduce и DuckDB:

Верификация singles: True

Верификация pairs: True

Верификация triples: True

Контрольные числа совпали полностью, расхождений не выявлено. Повторный прогон дал те же True × 3, что подтверждает детерминированность обоих методов.

Рост числа комбинаций (показан на реальных числах): уникальных одиночных навыков - 127, пар - 1018, троек - 5557. Отношения: пар к одиночным =(приблизительно) 8.0, троек к парам =(приблизительно) 5.5, троек к одиночным =(приблизительно) 43.8. Рост объясняется числом сочетаний C(n, k) = n! / (k!·(n−k)!) от числа n навыков внутри одной вакансии: при n = 5 одна вакансия порождает 5 одиночных записей, C(5,2) = 10 пар и C(5,3) = 10 троек, то есть 25 записей вместо 5. Доля пар и троек растёт нелинейно с увеличением n, поэтому агрегированное число уникальных пар и троек кратно превышает число одиночных навыков - это наблюдаемый эффект разветвления комбинаций, а не ошибка учёта.

5. ВЛИЯНИЕ НАВЫКОВ НА ЗАРПЛАТУ

Из 43 вакансий зарплата указана в 11. Глобальная медианная зарплата по выборке: 110 000 RUR. Учитывались навыки и комбинации, встречающиеся не менее 2 раз. Прибавка рассчитана относительно более частого базового навыка пары/тройки (по частоте из раздела 2).

Пары навыков, повышающие медианную зарплату:

(Entity Framework, MS SQL) | база: MS SQL | медиана базы: 105220 | медиана пары: 160000 | дельта: +54780

(C#, JavaScript) | база: C# | медиана базы: 112720 | медиана пары: 166470 | дельта: +53750

(.NET Framework, JavaScript) | база: .NET Framework | медиана базы: 155000 | медиана пары: 205000 | дельта: +50000

Пары навыков, не изменяющие / понижающие медианную зарплату:

(.NET Framework, ASP.NET) | база: ASP.NET | медиана базы: 160000 | медиана пары: 160000 | дельта: 0

(ASP.NET, Entity Framework) | база: ASP.NET | медиана базы: 160000 | медиана пары: 160000 | дельта: 0

(ASP.NET, MS SQL) | база: ASP.NET | медиана базы: 160000 | медиана пары: 160000 | дельта: 0

Тройки навыков: единственная тройка с положительной дельтой - (.NET Framework, Entity Framework, MS SQL), база .NET Framework, медиана базы 155 000, медиана тройки 160 000, дельта +5 000; остальные тройки дают дельту 0. Статистически значимых «понижающих» троек выборка не даёт: комбинации из трёх навыков редко повторяются ≥ 2 раз при 11 вакансиях с раскрытой компенсацией - методология верна, чисел для интерпретации «вниз» по тройкам датасет не содержит, что зафиксировано честно.

Сводка по одиночным навыкам (относительно глобальной медианы 110 000):

Повышают: JavaScript (+90 000), ORACLE (+57 500), C# (+50 000)

Понижают: 1С программирование (−45 000), MS SQL (−4 780); SQL - на уровне медианы (0)

Интерпретация. Связка Enterprise-стека (Entity Framework + MS SQL) и fullstack-связка (C# + JavaScript) дают прибавку 50–55 тыс. RUR к медиане базового навыка - это позиции с широкой зоной ответственности. Комбинации внутри чистого .NET/ASP.NET-стека не меняют медиану: рынок оценивает их как эквивалентные, прибавка «съедается» однородностью требований. Навык «1С программирование» в выборке C#-вакансий маркирует смежные роли интеграции/поддержки начального уровня, тяготеющие к нижней границе вилки, поэтому медиана по нему существенно ниже глобальной.

```
(.venv) PS C:\Users\hellboy\PycharmProjects\Data-Engineering-VK-2026\rustfs_lab> python salary_pairs_triples.py
Вакансий с ЗП: 11 | глобальная медиана: 110000
--- Пары навыков по зарплате ---
Вверх:
('Entity Framework', 'MS SQL') | база: MS SQL | медиана базы: 105220 | медиана combo: 160000 | дельта: 54780
('C#', 'JavaScript') | база: C# | медиана базы: 112720 | медиана combo: 166470 | дельта: 53750
('.NET Framework', 'JavaScript') | база: .NET Framework | медиана базы: 155000 | медиана combo: 205000 | дельта: 50000
Вниз:
('.NET Framework', 'ASP.NET') | база: ASP.NET | медиана базы: 160000 | медиана combo: 160000 | дельта: 0
('ASP.NET', 'Entity Framework') | база: ASP.NET | медиана базы: 160000 | медиана combo: 160000 | дельта: 0
('ASP.NET', 'MS SQL') | база: ASP.NET | медиана базы: 160000 | медиана combo: 160000 | дельта: 0
--- Тройки навыков по зарплате ---
Вверх:
('.NET Framework', 'Entity Framework', 'MS SQL') | база: .NET Framework | медиана базы: 155000 | медиана combo: 160000 | дельта: 5000
('.NET Framework', 'ASP.NET', 'Entity Framework') | база: ASP.NET | медиана базы: 160000 | медиана combo: 160000 | дельта: 0
('.NET Framework', 'ASP.NET', 'MS SQL') | база: ASP.NET | медиана базы: 160000 | медиана combo: 160000 | дельта: 0
Вниз:
('.NET Framework', 'ASP.NET', 'Entity Framework') | база: ASP.NET | медиана базы: 160000 | медиана combo: 160000 | дельта: 0
('.NET Framework', 'ASP.NET', 'MS SQL') | база: ASP.NET | медиана базы: 160000 | медиана combo: 160000 | дельта: 0
('ASP.NET', 'Entity Framework', 'MS SQL') | база: ASP.NET | медиана базы: 160000 | медиана combo: 160000 | дельта: 0
```

6. РЕГИОНАЛЬНЫЙ АНАЛИЗ

Выбраны три региона с наибольшим числом вакансий варианта среди имеющих раскрытую компенсацию (чтобы медиана была корректной во всех строках обеих таблиц).

Таблица 1. Зарплаты по регионам:

Регион | вакансий | с ЗП | медиана ЗП

Москва | 18 | 2 | 125000

Санкт-Петербург | 8 | 1 | 100440

Воронеж | 2 | 1 | 90000

Таблица 2. Топ-5 навыков по регионам:

Москва | Санкт-Петербург | Воронеж

ASP.NET : 5 | IT : 3 | 1С программирование : 2

.NET Framework : 4 | PostgreSQL : 3 | SQL : 1

RabbitMQ : 4 | Разработка логистики : 3 | 1С: Бухгалтерия : 1

SQL : 3 | C# : 3 | 1С: Зарплата и кадры : 1

PostgreSQL : 3 | C# : 2 | 1С: Бухгалтерия и склад : 1

Инсайт. Москва даёт и наибольший объём (18 вакансий), и наибольшую медиану (125 000), причём требования сформулированы конкретно по фреймворкам и инфраструктуре (ASP.NET, .NET Framework, RabbitMQ) - рынок зрелый, работодатели точно знают стек. Санкт-Петербург уступает по медиане ~20%, а в топе навыков оказывается размытое «IT» наряду с предметными доменами (логистика, ПВЗ) - больше общих/младших позиций. Воронеж выпадает из .NET-контекста: топ навыков занят 1С, что маркирует региональный рынок как рынок сопровождения/интеграции с низкой раскрытостью компенсации. Тренд подтверждает концентрацию высокооплачиваемых C#/.NET-ролей в столицах и их размывание в регионах.

```
(.venv) PS C:\Users\hellboy\PycharmProjects\Data-Engineering-VK-2026\rustfs_lab> python regions_fixed.py
Зарплаты по регионам
Регион | вакансий | с ЗП | медиана ЗП
Москва | 18 | 2 | 125000
Санкт-Петербург | 8 | 1 | 100440
Воронеж | 2 | 1 | 90000
Топ-5 навыков по регионам
Регион: Москва
    ASP.NET : 5
    .NET Framework : 4
    RabbitMQ : 4
    SQL : 3
    PostgreSQL : 3
Регион: Санкт-Петербург
    IT : 3
    PostgreSQL : 3
    Разработка логистики : 3
    C# : 3
    C# : 2
Регион: Воронеж
    1С программирование : 2
    SQL : 1
    1С: Бухгалтерия : 1
    1С: Зарплата и кадры : 1
    1С: Бухгалтерия и склад : 1
```

7. СРАВНЕНИЕ ПОДХОДОВ ПО ВРЕМЕНИ И ПАМЯТИ

Метрика | MapReduce (из S3) | DuckDB (локально)

Среднее время (3 попытки) | 1.12 сек | 0.1332 сек

Средний пик памяти | 0.94 MB | 1.85 MB

DuckDB быстрее примерно в 8 раз (1.12 / 0.1332 =(приблизительно) 8.4): он читает данные локально и агрегирует в колоночном движке без сетевого оверхеда. MapReduce медленнее, потому что платит сетевой I/O к объектному хранилищу (чанки по 10 объектов через S3 API). Зато MapReduce экономнее по памяти примерно в 2 раза (0.94 MB против 1.85 MB): он обрабатывает данные потоково чанками и не материализует весь датасет в

оперативной памяти Python, тогда как DuckDB грузит полный DataFrame и строит колоночные структуры. Оба подхода дают идентичные контрольные числа (раздел 4), что подтверждает эквивалентность методик при разной физической реализации. Для архивного WORM-датасета с частым чтением MapReduce поверх S3 оправдан горизонтальным масштабированием и отказоустойчивостью EC; DuckDB оправдан для интерактивной аналитики на локализованном срезе.

8. ЗАКЛЮЧЕНИЕ

Аналитический пайплайн построен end-to-end: объектное хранилище RustFS с Erasure Coding, загрузка датасета, фильтрация по индивидуальному варианту C#/.NET (43 вакансии), расчёт частот навыков, пар и троек двумя независимыми методами (собственный MapReduce из S3 и SQL в DuckDB с самостоятельно записанным условием варианта) с полным совпадением контрольных чисел, обработка отказа маппера проверена прогоном, профилирование в три попытки по времени и пиковой памяти для обоих методов, анализ влияния навыков на медианную зарплату (пары, тройки и одиночные навыки с прибавкой от базового навыка) и сравнительный региональный разбор по двум таблицам. Рост числа пар и троек показан на реальных числах (127 / 1018 / 5557) и объяснён комбинаторикой сочетаний внутри вакансии. Результаты трёх уровней стека лежат в бакете hh-lake. Исходный код и отчёт опубликованы в репозитории GitHub.

Репозиторий: https://github.com/fromhellboyyy/Data-Engineering-VK-2026